Day 8 - Lead Engineer System Design + LLD Playbook

For most interviews, your goal is not to invent the perfect architecture. Your goal is to show a **repeatable decision-making process**:

```
Requirements
    ↓
Scale assumptions
    ↓
API contracts
    ↓
Data model
    ↓
Architecture
    ↓
Critical deep dive
    ↓
Trade-offs and summary
```

The interviewer should feel:

> "This candidate can take an unclear business problem, turn it into engineering requirements, and design a reliable production system."

A. The Six-Phase System Design Framework

Phase 1: Clarify requirements

Do not start drawing services immediately.

First determine:

- Who uses the system?
- What is the main user journey?
- What is included in the MVP?
- What is explicitly out of scope?
- What scale and reliability are expected?

Separate requirements into two categories.

Functional requirements

These describe **what the system does**.

For an AgentRun service:

- Start an agent workflow.
- Retrieve current run status.
- List previous runs.
- Receive asynchronous tool results.
- Cancel a running workflow.
- Store output artifacts.
- Record audit events.

Non-functional requirements

These describe **how well the system must operate**.

Examples:

- `GET run status` should have p95 latency under 200 ms.
- Starting a run should be available 99.9% of the time.
- A submitted run must not be silently lost.
- Duplicate requests must not create duplicate runs.
- Tenant data must remain isolated.
- Audit records must be retained for one year.
- The platform should support horizontal scaling.

Interview language

Say:

> "Before designing the architecture, I would like to confirm the main user flow, expected scale, consistency requirements, and what belongs in the MVP."

Phase 2: Capacity estimation

Capacity estimation tells you whether you need:

- One application and one database
- Caching
- Partitioning
- Async queues
- Object storage
- Batch processing
- Multi-region architecture

You do not need exact numbers. You need **reasonable assumptions and correct formulas**.

Always estimate at least:

- Requests per day
- Average QPS
- Peak QPS
- Data generated per day
- Bandwidth
- Retention storage

For AI systems, also consider:

- LLM requests per second
- Tokens per second
- Tool calls per run
- Embedding throughput
- Vector count
- Artifact size

Phase 3: Define API contracts

The APIs clarify:

- System boundaries
- Synchronous versus asynchronous behavior
- Resource ownership
- Idempotency
- Pagination
- Error semantics
- Authentication requirements

For long-running work, prefer:

```
Client starts work
        ↓
API returns 202 Accepted + run_id
        ↓
Work runs asynchronously
        ↓
Client polls, subscribes or receives webhook
```

Do not hold an HTTP connection open for a 20-minute agent workflow unless streaming is an explicit requirement.

Phase 4: Define the data model

Start with the main entities and relationships.

For every major table or collection, identify:

- Primary key
- Tenant or ownership key
- Foreign keys
- Status
- Timestamps
- Query patterns
- Important indexes
- Data retention policy

Do not design indexes without first stating the queries.

The correct thought process is:

```
Query:
Last 20 runs for a project

Required fields:
project_id, created_at, run_id, status

Index:
(project_id, created_at DESC, run_id DESC)
```

Phase 5: Draw the high-level architecture

Show:

- Clients
- API gateway
- Authentication
- Core services
- Databases
- Cache
- Queue or event bus
- Workers
- Object storage
- External integrations
- Observability

Explain the main request flow before discussing every box.

Phase 6: Deep dive, trade-offs and summary

Pick one or two important areas rather than shallowly discussing everything.

For an agent platform, good deep dives include:

- Run state management
- Idempotency
- Workflow recovery
- Tool callback handling
- Queue delivery semantics
- Multi-tenant isolation
- Audit logging
- LLM and tool cost controls

End by summarizing:

1. Main architecture
2. Reliability model
3. Scaling strategy
4. Important trade-off
5. Future extension

Reusable Clarifying Questions

You do not need to ask all 20. Select the questions that materially affect the design.

Product and scope

1. Who are the primary users or calling systems?
2. What is the main user journey?
3. What functionality is required for the MVP?
4. What should be explicitly out of scope?
5. Is the operation synchronous, asynchronous or streaming?
6. Can users cancel, retry, pause or resume an operation?
7. What are the possible lifecycle states?
8. Are users allowed to access historical results?

Scale and performance

1. How many daily active users or API clients are expected?
2. How many operations does each user perform per day?
3. What is the expected peak-to-average traffic ratio?
4. What are the typical and maximum request payload sizes?
5. What are the p50, p95 and p99 latency expectations?

Reliability and consistency

1. What availability target is required?
2. Can the system temporarily return stale status information?
3. What happens if the same request is submitted twice?
4. Can events arrive more than once or out of order?
5. What are the recovery-time and recovery-point objectives?

Security and operations

1. Is the platform multi-tenant, and how strong must tenant isolation be?
2. Are there compliance, audit, encryption, data-residency, cost or integration constraints?

A useful final scope statement is:

> “For the MVP, I will design an asynchronous, multi-tenant AgentRun service supporting start, status, listing, callbacks and cancellation. I will treat workflow authoring, model training and advanced human approval as phase-two features.”

B. Capacity Estimation

Core formulas

Daily operations

```
Operations/day = DAU × operations per user per day
```

Example:

```
100,000 DAU × 4 runs/day = 400,000 runs/day
```

Average QPS

```
Average QPS = requests per day ÷ 86,400
```

Interview approximation:

```
1 million requests/day ≈ 11.6 average QPS
```

Peak QPS

```
Peak QPS = average QPS × peak factor
```

Typical assumed peak factors:

- Even machine traffic: 2–3×
- Normal business traffic: 5×
- Highly bursty consumer traffic: 10× or more

Always state that the factor is an assumption.

Network bandwidth

```
Bandwidth per second = QPS × average payload size
```

For daily traffic:

```
Daily transfer = requests/day × payload size
```

Remember to include both request and response payloads when relevant.

Storage growth

```
Storage/day =
    objects created/day
    × average object size
```

Then add:

```
Logical storage =
    base data
    + index overhead
    + metadata
```

Physical storage may also include:

```
Physical storage =
    logical storage
    × replication factor
```

Retention

```
Retention storage = storage/day × retention days
```

Cache impact

```
Database read QPS =
    application read QPS
    × (1 − cache hit rate)
```

More generally:

```
Backend QPS =
    writes
    + reads × (1 − cache hit rate)
```

Do not assume a cache hit rate without explaining why the data is reusable.

Mini Scenario 1: AgentRun API

Assumptions

- 100,000 DAU
- Four runs per active user per day
- Six status reads per run
- Five tool callbacks per run
- Peak traffic is 10× average
- Start request plus response: 5 KB
- Status request plus response: 2.5 KB
- Tool callback request plus response: 8.5 KB

Run volume

```
100,000 × 4 = 400,000 runs/day
```

Request traffic

Operation	Requests/day	Average QPS	Peak QPS
Start run	400,000	4.6	46
Status reads	2,400,000	27.8	278
Tool callbacks	2,000,000	23.1	231
Total	**4,800,000**	**55.6**	**556**

This is not enormous traffic for an API layer. The harder scaling problem is likely to be:

- Agent workflow execution
- External tool latency
- LLM token throughput

- Long-running state
- Retry behavior

That distinction is valuable in an interview.

Bandwidth

Operation	Calculation	Transfer/day
Start run	400K × 5 KB	2 GB
Status	2.4M × 2.5 KB	6 GB
Callbacks	2M × 8.5 KB	17 GB
Total		**25 GB/day**

Average external bandwidth:

```
25 GB × 8 ÷ 86,400
≈ 2.3 Mbps average
```

Approximate peak:

```
2.3 × 10 ≈ 23 Mbps
```

This excludes potentially much larger internal LLM, tool and artifact traffic.

Structured storage

Assume each run creates:

Data	Approximate size/run	Daily storage
AgentRun row	4 KB	1.6 GB
Five ToolExecution rows	15 KB	6 GB
Audit events	12 KB	4.8 GB
Artifact metadata	2 KB	0.8 GB
Total structured data	**33 KB**	**13.2 GB/day**

Add approximately 30% for indexes and database overhead:

```
13.2 GB × 1.3 ≈ 17.2 GB/day logical
```

For one-year retention:

```
17.2 × 365 ≈ 6.3 TB
```

Large artifact bodies should live in object storage, not inside the relational database.

Cache effect

Status reads average about 28 QPS.

With a 70% status-cache hit rate:

```
Database status-read QPS =
28 × (1 − 0.70)
≈ 8.4 QPS
```

The cache should be updated or invalidated when the run status changes.

Mini Scenario 2: Document Ingestion

Assumptions

- 20,000 PDFs per day
- Average PDF size: 8 MB
- Average 40 pages per PDF
- Average 500 chunks per PDF
- Chunk text: 2 KB
- Embedding dimension: 1,536
- Float32 embedding: four bytes per dimension
- Metadata: 0.5 KB per chunk
- Peak load: 10× average

Document throughput

```
20,000 ÷ 86,400
≈ 0.23 documents/second average
```

Peak:

```
0.23 × 10
≈ 2.3 documents/second
```

Page throughput

```
20,000 × 40 = 800,000 pages/day
```

```
800,000 ÷ 86,400
≈ 9.3 pages/second average
```

Peak:

```
≈ 93 pages/second
```

Chunk and embedding throughput

```
20,000 × 500
= 10 million chunks/day
```

```
10 million ÷ 86,400
≈ 116 chunks/second average
```

Peak:

```
≈ 1,160 chunks/second
```

With batches of 64 embeddings:

```
10,000,000 ÷ 64
= 156,250 embedding batches/day
```

```
156,250 ÷ 86,400
≈ 1.8 embedding requests/second average
```

Peak:

```
≈ 18 embedding requests/second
```

In a real interview, also estimate **tokens per second**, because embedding providers often enforce token-based limits rather than only request limits.

Storage

Raw PDF storage

```
20,000 × 8 MB = 160 GB/day
```

Chunk text

```
10 million × 2 KB = 20 GB/day
```

Embeddings

One vector:

```
1,536 dimensions × 4 bytes
= 6,144 bytes
≈ 6 KB
```

Daily vectors:

```
10 million × 6 KB
≈ 60 GB/day
```

More precisely, it is approximately 61.4 GB/day.

Chunk metadata

```
10 million × 0.5 KB = 5 GB/day
```

Vector index overhead

Assume 30%:

```
61.4 GB × 30%
≈ 18.4 GB/day
```

Total

Storage component	Growth/day
Raw PDFs	160 GB
Chunk text	20 GB
Embeddings	61.4 GB
Metadata	5 GB
Vector-index overhead	18.4 GB
Total	**264.8 GB/day**

This immediately suggests:

- Object storage for PDFs
- Async ingestion
- Horizontally scalable parsing workers
- Batched embedding generation
- Vector-store capacity planning
- Deduplication using document checksum
- Lifecycle and retention policies

C. API Design Expectations

General REST expectations

A strong API design should demonstrate:

- Resource-oriented URLs
- Appropriate HTTP methods
- Versioning
- Request validation
- Authentication and authorization
- Idempotency
- Pagination
- Filtering
- Stable error format
- Async-operation semantics

Use nouns rather than action-heavy paths where practical:

```
POST /v1/projects/{project_id}/runs
GET  /v1/runs/{run_id}
GET  /v1/projects/{project_id}/runs
```

An action endpoint is acceptable for state transitions:

```
POST /v1/runs/{run_id}:cancel
```

Pagination

Avoid offset pagination for large or frequently changing tables:

```
?page=10000&size=20
```

Problems:

- Large offsets become expensive.
- New rows can cause duplicates or missing results.
- Results become unstable while users paginate.

Prefer cursor or keyset pagination:

```
GET /v1/projects/p123/runs
    ?limit=20
    &cursor=eyJjcmVhdGVkX2F0IjoiLi4uIiwicnVuX2lkIjoiLi4uIn0
```

Internally:

```
WHERE project_id = :project_id
  AND (created_at, run_id) < (:cursor_time, :cursor_run_id)
ORDER BY created_at DESC, run_id DESC
LIMIT 20;
```

The `run_id` acts as a tie-breaker when multiple runs have the same timestamp.

Idempotency

Use idempotency for operations where retries could produce unwanted duplicates.

Important examples:

- Starting a run
- Creating a document-ingestion job
- Processing tool callbacks
- Billing-related operations

Client sends:

```
Idempotency-Key: 5e15804c-9c89-41fa-9e4c-f021f55fe5b8
```

The server stores something similar to:

```
tenant_id
idempotency_key
request_hash
resource_id
response_code
response_body
expires_at
```

Behavior:

1. First request creates the run.
2. Retry with the same key and same payload returns the original result.
3. Same key with a different payload returns a conflict.
4. Idempotency should normally be scoped to a tenant or account.

Standard error model

```
{
  "error": {
    "code": "RUN_NOT_FOUND",
    "message": "The requested agent run was not found.",
    "request_id": "req_01K1A3EXAMPLE",
    "details": {
      "run_id": "run_01K1A3EXAMPLE"
    }
  }
}
```

Recommended fields:

- `code`: stable machine-readable code
- `message`: safe human-readable explanation
- `request_id`: correlation identifier
- `details`: optional structured context

Do not expose:

- Stack traces
- Database errors
- Secrets
- Internal hostnames
- Tool credentials

D. Agent Platform Data Model

Main entities

Tenant

```
Tenant
- tenant_id PK
- name
- status
- plan
- region
- created_at
```

User

```
User
- user_id PK
- tenant_id FK
- email
- status
- created_at
```

Project

```
Project
- project_id PK
- tenant_id FK
- name
- status
- created_at
```

AgentRun

```
AgentRun
- run_id PK
- tenant_id FK
- project_id FK
- agent_id
- agent_version
- status
- idempotency_key
- input_uri or encrypted_input
- current_step
- version
- error_code
- created_by
- created_at
- started_at
- completed_at
- updated_at
```

The `version` field can support optimistic concurrency during state transitions.

ToolExecution

```
ToolExecution
- tool_execution_id PK
- tenant_id FK
- run_id FK
- tool_type
- tool_version
- status
- attempt_number
- callback_event_id
- request_uri
- response_uri
- error_code
- created_at
- started_at
- completed_at
```

Artifact

```
Artifact
- artifact_id PK
- tenant_id FK
- run_id FK
- artifact_type
- object_uri
- checksum
- content_type
- size_bytes
- created_at
```

Feedback

```
Feedback
- feedback_id PK
- tenant_id FK
- run_id FK
- user_id FK
- rating
- comment
- created_at
```

AuditLog

```
AuditLog
- event_id PK
- tenant_id
- project_id
- run_id
- actor_type
- actor_id
- event_type
- resource_type
- resource_id
- event_payload
- request_id
- occurred_at
```

For high-volume audit events, this may eventually move to a dedicated append-only store.

Five important indexes

1. Last 20 runs for a project

```
CREATE INDEX idx_agent_runs_project_created
ON agent_runs (
    project_id,
    created_at DESC,
    run_id DESC
);
```

Supports:

```
Last 20 AgentRuns for a project
```

2. Audit trail by tenant and time

```
CREATE INDEX idx_audit_tenant_time
ON audit_logs (
    tenant_id,
    occurred_at DESC,
    event_id DESC
);
```

Supports:

```
Audit events for tenant X between two timestamps
```

3. Tool failures by type and status

```
CREATE INDEX idx_tool_failure_type_status_time
ON tool_executions (
    tenant_id,
    tool_type,
    status,
    created_at DESC
);
```

A partial index may be even smaller:

```
CREATE INDEX idx_failed_tool_executions
ON tool_executions (
    tenant_id,
    tool_type,
    created_at DESC
)
WHERE status = 'FAILED';
```

4. Start-run idempotency

```
CREATE UNIQUE INDEX idx_run_idempotency
ON agent_runs (
    tenant_id,
    idempotency_key
);
```

5. Tool executions for a run

```
CREATE INDEX idx_tool_executions_run_time
ON tool_executions (
    run_id,
    created_at ASC
);
```

Also consider unique callback-event IDs to deduplicate callbacks:

```
UNIQUE (tenant_id, callback_event_id)
```

E. LLD Basics

Responsibility

A class or component should have a clear reason to exist.

Bad:

```
AgentManager
- validates HTTP
- queries database
- calls tools
- sends emails
- calculates billing
- formats logs
```

Better:

```
RunController        → HTTP concerns
RunService           → use-case orchestration
RunRepository        → persistence
RunStateMachine      → valid transitions
ToolCallbackService  → callback processing
EventPublisher       → event delivery
```

Cohesion

High cohesion means related behavior stays together.

For example:

```
RunStateMachine
- can_transition()
- transition()
- is_terminal()
```

All methods concern run lifecycle rules.

Coupling

Low coupling means components depend on small interfaces rather than concrete infrastructure.

Bad:

```
class RunService:
    def __init__(self):
        self.db = PostgreSQLConnection(...)
        self.kafka = KafkaProducer(...)
```

Better conceptually:

```
class RunService:
    def __init__(
        self,
        repository: RunRepository,
        publisher: EventPublisher,
    ):
        self.repository = repository
        self.publisher = publisher
```

The service does not care whether the implementation uses PostgreSQL, DynamoDB, Kafka or another technology.

Interfaces

Interfaces define what a dependency can do without exposing how it does it.

```
from typing import Protocol


class RunRepository(Protocol):
    def create(self, run: "AgentRun") -> None:
        ...

    def get(self, run_id: str, tenant_id: str) -> "AgentRun | None":
        ...

    def update_status(
        self,
        run_id: str,
        expected_version: int,
        new_status: str,
    ) -> bool:
        ...


class EventPublisher(Protocol):
    def publish_run_requested(self, run_id: str) -> None:
        ...
```

Reusable LLD template

When asked to deep dive a component, follow this order:

1. Classes and responsibilities

```
RunController
RunService
RunRepository
RunStateMachine
IdempotencyStore
EventPublisher
AuditWriter
```

2. Interfaces

Define the contracts between components.

3. Sequence flow

Explain which object calls which object.

4. State and concurrency

Explain:

- Status transitions
- Locks or optimistic concurrency
- Duplicate requests
- Transaction boundaries

5. Edge cases

Explain:

- Missing run
- Duplicate callback
- Invalid transition
- Timeout
- Partial failure
- Unauthorized tenant access

6. Tests

Include:

- Unit tests
- Repository integration tests
- Contract tests
- Concurrency tests
- Failure-injection tests

F. Case Study: AgentRun Service

Problem statement

Design a service that:

- Starts an agent workflow

- Tracks its status
- Executes external tools asynchronously
- Stores results and artifacts
- Supports auditability

Deliverable 1: Functional and NFR Requirements

MVP functional requirements

1. Start an AgentRun for a project.
2. Return a unique `run_id`.
3. Track lifecycle status.
4. Retrieve the current run status.
5. List runs for a project.
6. Execute workflow steps asynchronously.
7. Receive asynchronous tool callbacks.
8. Retry transient workflow and tool failures.
9. Cancel a non-terminal run.
10. Store output-artifact metadata.
11. Record security and lifecycle audit events.
12. Enforce tenant-level authorization.

Phase-two functional requirements

1. Pause and resume workflows.
2. Human approval steps.
3. Server-Sent Events or WebSocket streaming.
4. Customer webhooks for status changes.
5. Scheduled runs.
6. Workflow replay from checkpoints.
7. Agent-version comparison.
8. Per-tenant quotas and budgets.
9. Multi-region execution.
10. Advanced run analytics and evaluation.

MVP NFR assumptions

Area	Assumption
Start-run latency	p95 under 300 ms, excluding workflow completion
Status latency	p95 under 200 ms
Availability	99.9%
Durability	Accepted runs must not be silently lost
Delivery	At-least-once event delivery
Consistency	Strong for creation; eventual for cached status
Scale	Approximately 556 peak external QPS
Isolation	All records scoped and authorized by tenant
Encryption	TLS in transit; encryption at rest
Audit retention	One year
Run retention	30 days hot, longer-term archive configurable
Recovery	Failed workers resume from persisted state
Observability	Metrics, logs and traces correlated by request/run ID

Important distinction:

> At-least-once delivery means messages can be duplicated. Therefore, consumers must be idempotent.

Deliverable 2: Capacity Table

Using the earlier assumptions:

Metric	Average	Peak or daily total
Runs	4.6 runs/sec	46 runs/sec
External API traffic	55.6 QPS	556 QPS
Status reads	27.8 QPS	278 QPS
Tool callbacks	23.1 QPS	231 QPS
External bandwidth	2.3 Mbps	Approximately 23 Mbps
Structured storage	—	13.2 GB/day
Structured storage with indexes	—	Approximately 17.2 GB/day
One-year structured retention	—	Approximately 6.3 TB
Tool executions	23/sec	231/sec
Audit events	55.6/sec	Approximately 556/sec

This table excludes:

- Artifact bodies
- LLM request and response tokens
- Tool-specific network transfer
- Replication overhead
- Backup storage

Call out exclusions explicitly during the interview.

Deliverable 3: Eight APIs

1. Start a run

```
POST /v1/projects/{project_id}/runs
Authorization: Bearer <token>
Idempotency-Key: <unique-key>
Content-Type: application/json
```

```
{
  "agent_id": "support-agent",
  "agent_version": "v12",
  "input": {
    "question": "Why did deployment 782 fail?"
  },
  "configuration": {
    "max_steps": 20,
    "timeout_seconds": 900
  }
}
```

Response:

```
202 Accepted
```

```
{
  "run_id": "run_01K1EXAMPLE",
  "status": "QUEUED",
  "created_at": "2026-07-28T12:00:00Z",
  "status_url": "/v1/runs/run_01K1EXAMPLE"
}
```

---

2. Get run status

```
GET /v1/runs/{run_id}
```

```
{
  "run_id": "run_01K1EXAMPLE",
  "project_id": "project_123",
  "status": "WAITING_FOR_TOOL",
  "current_step": "fetch_pipeline_logs",
  "progress": {
    "completed_steps": 3,
    "total_steps_estimate": 7
  },
  "created_at": "2026-07-28T12:00:00Z",
  "updated_at": "2026-07-28T12:00:14Z"
}
```

The progress estimate should be optional because agent workflows are not always predictable.

---

3. List project runs

```
GET /v1/projects/{project_id}/runs
    ?status=FAILED
    &agent_id=support-agent
    &created_after=2026-07-01T00:00:00Z
    &limit=20
    &cursor=<opaque-cursor>
```

Response:

```
{
  "items": [],
  "next_cursor": "opaque-next-cursor",
  "has_more": true
}
```

---

4. Cancel a run

```
POST /v1/runs/{run_id}:cancel
Idempotency-Key: <unique-key>
```

```
{
  "run_id": "run_01K1EXAMPLE",
  "status": "CANCELLATION_REQUESTED"
}
```

Cancellation is normally cooperative rather than instantaneous.

---

5. Tool callback

```
POST /v1/runs/{run_id}/tool-executions/{tool_execution_id}/callback
X-Callback-Signature: <signature>
```

```
{
  "event_id": "event_456",
  "status": "SUCCEEDED",
  "attempt": 1,
  "output_uri": "s3://agent-artifacts/tenant-a/output.json",
  "completed_at": "2026-07-28T12:01:00Z"
}
```

The `event_id` should be unique and deduplicated.

---

6. Ingest a document

This should normally be owned by a separate Document Service even if exposed through the same API gateway.

```
POST /v1/projects/{project_id}/documents
Idempotency-Key: <unique-key>
Content-Type: multipart/form-data
```

Response:

```
202 Accepted
```

```
{
  "document_id": "doc_123",
  "ingestion_job_id": "ingest_456",
  "status": "QUEUED"
}
```

---

7. Search audit events

```
GET /v1/tenants/{tenant_id}/audit-events
    ?from=2026-07-01T00:00:00Z
    &to=2026-07-28T23:59:59Z
    &event_type=TOOL_EXECUTION_FAILED
    &limit=100
    &cursor=<opaque-cursor>
```

This endpoint needs strict role-based access control.

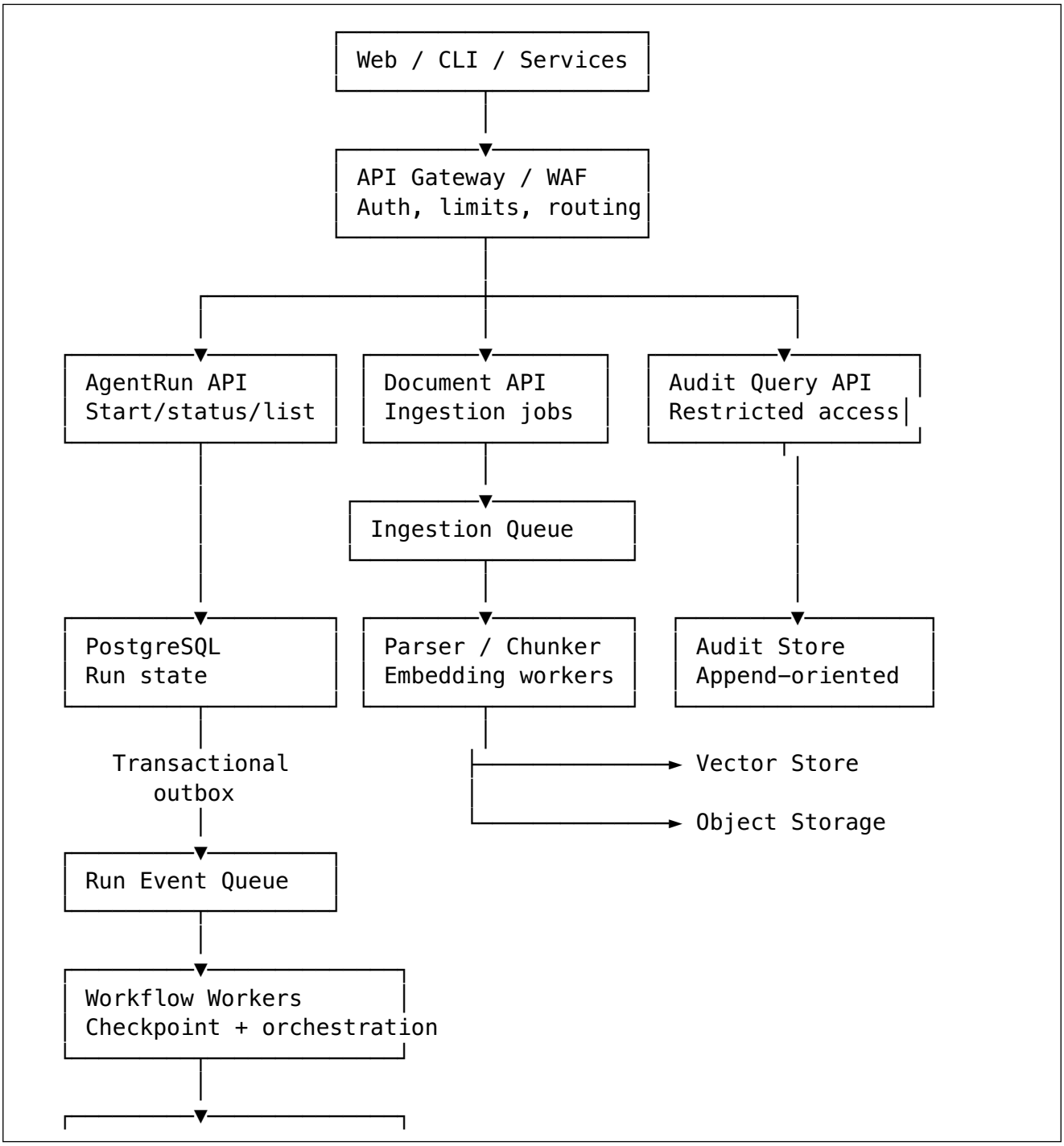
---

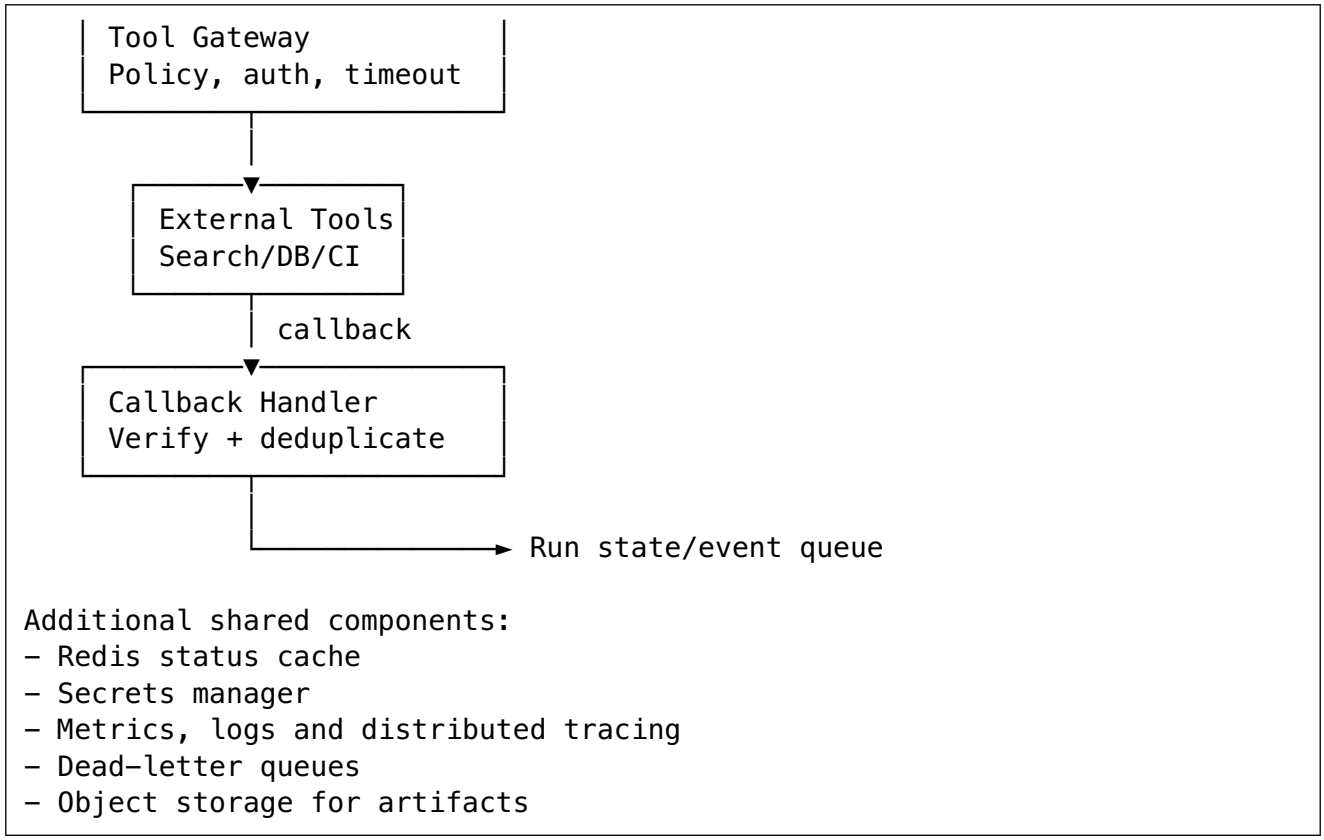
8. Submit run feedback

```
POST /v1/runs/{run_id}/feedback
Idempotency-Key: <unique-key>
```

```
{
  "rating": 4,
  "comment": "The diagnosis was correct but too verbose."
}
```

Deliverable 4: Architecture






```
Additional shared components:
- Redis status cache
- Secrets manager
- Metrics, logs and distributed tracing
- Dead-letter queues
- Object storage for artifacts
```

Main start-run flow

1. Client calls `POST /runs` with an idempotency key.
2. API gateway authenticates the caller.
3. AgentRun service checks tenant and project permissions.
4. Service validates the agent version and input.
5. In one database transaction:

- Create the AgentRun with `QUEUED` status.
- Store the idempotency record.
- Insert a `RUN_REQUESTED` outbox event. 6. Outbox publisher sends the event to the queue. 7. Workflow worker claims the run. 8. Worker changes status to `RUNNING`. 9. Worker checkpoints after important steps. 10. Tool requests pass through the Tool Gateway. 11. Tool callbacks resume the workflow. 12. Final result and artifact references are stored. 13. Run becomes `SUCCEEDED`, `FAILED`, `TIMED_OUT` or `CANCELLED`.

Why use a transactional outbox?

A dangerous implementation is:

```
1. Insert run into database
2. Publish queue message
```

Suppose step one succeeds and step two fails. The API returns a run ID, but no worker ever receives it.

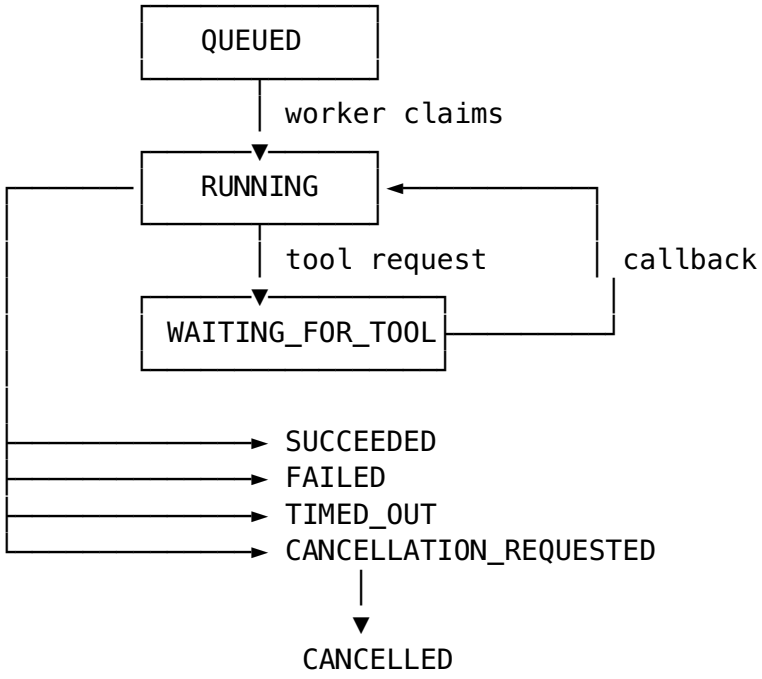
With an outbox:

```
One database transaction:
- Insert AgentRun
- Insert OutboxEvent
```

A background publisher repeatedly sends unpublished outbox events.

This provides reliable handoff without requiring a distributed transaction between the database and message broker.

Run state machine



Prevent arbitrary updates such as:

```
SUCCEEDED → RUNNING
FAILED → WAITING_FOR_TOOL
CANCELLED → SUCCEEDED
```

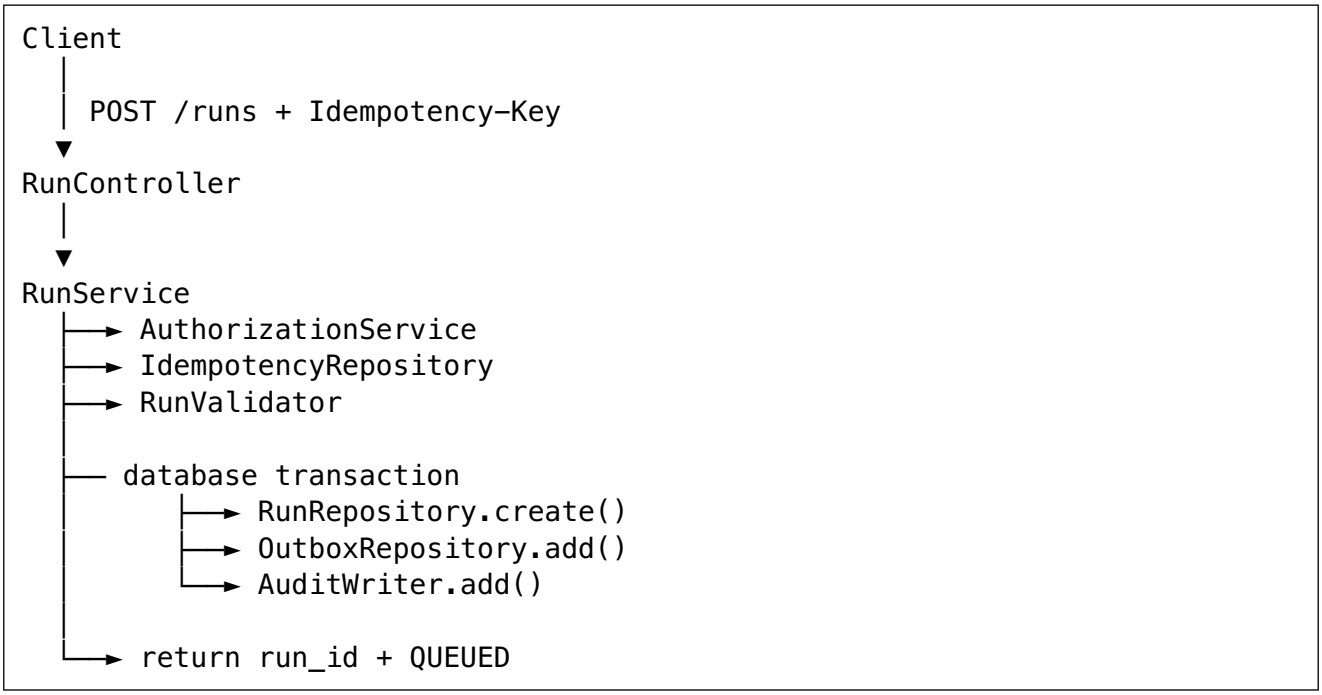
Use a state-machine component plus optimistic concurrency.

LLD Deep Dive: Starting a Run

Classes

```
RunController
- Parses HTTP request
- Returns HTTP response
- Maps exceptions to error codes

RunService
- Implements the start-run use case
- Coordinates authorization, idempotency and persistence

RunValidator
- Validates agent configuration and input

RunStateMachine
- Defines valid states and transitions

RunRepository
- Stores and retrieves AgentRuns

IdempotencyRepository
- Stores idempotency request and response records

OutboxRepository
- Writes transactional events

AuthorizationService
- Checks tenant/project permissions

AuditWriter
- Creates security and business audit events
```

Simplified sequence



Important edge cases

1. Same idempotency key and same payload:
   - Return the existing run.
1. Same idempotency key and different payload:
   - Return `409 IDEMPOTENCY_KEY_REUSED`.
1. Project does not belong to tenant:
   - Return `404` or authorization-safe response.
1. Invalid agent version:
   - Return `422 AGENT_VERSION_INVALID`.
1. Database transaction fails:
   - No run and no outbox event should remain.
1. Queue is temporarily unavailable:
   - Run remains queued; outbox publisher retries.
1. Worker processes the same event twice:
   - Claim or state update must be idempotent.
1. Two workers attempt to claim the same run:
   - Use conditional update or optimistic concurrency.
1. Callback arrives before worker status update:
   - Persist and reconcile, or reject with retryable semantics.
1. Callback arrives after run cancellation:
   - Record the event but do not reactivate the run.

Testing approach

Unit tests

- Input validation
- Idempotency behavior
- Authorization behavior
- State transitions
- Error mapping

Integration tests

- Database transaction rollback
- Unique-idempotency constraint
- Outbox publishing
- Repository indexes and queries

Concurrency tests

- Two start requests with the same idempotency key
- Two workers claiming the same run
- Callback and cancellation arriving simultaneously

Failure-injection tests

- Queue unavailable
- Database timeout
- Worker crashes after tool execution
- Duplicate callback
- Tool timeout
- Object-storage failure

Reliability, Security and Observability Deep Dives

Reliability

Use:

- Transactional outbox
- At-least-once queues
- Idempotent consumers
- Exponential backoff with jitter
- Maximum retry limits
- Dead-letter queues
- Worker heartbeats or leases
- Durable workflow checkpoints
- Timeouts for every external call
- Circuit breakers for unhealthy tools

Do not claim “exactly once” casually. In distributed systems, it is usually safer to say:

> “The transport is at-least-once, and the business operation behaves effectively once through idempotency and deduplication.”

Multi-tenant security

Every request should derive the tenant from authenticated identity rather than blindly trusting a body field.

Enforce tenant isolation through:

- Tenant-aware authorization
- Tenant key in database queries
- Row-level security where appropriate
- Tenant-specific quotas
- Tenant-specific encryption keys for stricter environments
- Signed tool callbacks
- Short-lived tool credentials

- Secrets manager
- Artifact URLs with limited duration

The Tool Gateway should validate:

- Is this tool allowed for this tenant?
- Is the requested operation allowed?
- What credentials should be used?
- What timeout applies?
- Is human approval required?
- What data may leave the platform?

Observability

Use three identifiers consistently:

```
request_id
run_id
tool_execution_id
```

Metrics

- Run starts per second
- Run completion rate
- Run failure rate by agent version
- Queue age
- Time spent in each state
- Tool failure rate
- Tool latency
- LLM latency and token usage
- Cost per run
- Retry counts
- Dead-letter queue size
- Status-cache hit rate

Logs

Use structured logs:

```
{
  "level": "ERROR",
  "event": "tool_execution_failed",
  "tenant_id": "tenant_123",
  "run_id": "run_456",
  "tool_execution_id": "tool_789",
  "tool_type": "pipeline_log_fetch",
  "error_code": "TOOL_TIMEOUT",
  "request_id": "req_111"
}
```

Tracing

One distributed trace can connect:

```
Start API
→ Queue publication
→ Workflow worker
→ LLM call
→ Tool execution
→ Callback
→ Final persistence
```

Key Trade-offs

PostgreSQL versus NoSQL

Start with PostgreSQL when you need:

- Transactions
- Idempotency constraints
- Relationships
- Flexible indexed queries
- Strong state-transition consistency

Consider NoSQL or partitioned stores when:

- Run volume becomes extremely large
- Access patterns are narrow and predictable
- Global active-active writes are required
- Individual tenant partitions need independent scaling

A good interview answer:

> “I would begin with PostgreSQL because run creation, state transitions and idempotency benefit from transactions. I would partition by time or tenant as volume grows, and move append-heavy audit events to a separate store if they dominate database load.”

Polling versus streaming

Polling

Advantages:

- Simple
- Easy to retry
- Works through most networks

Disadvantages:

- Repeated requests
- Delayed updates
- Wasteful for long runs

SSE

Advantages:

- Simple server-to-client streaming
- Good for status and token updates
- Easier than WebSockets for one-way events

Disadvantages:

- Long-lived connections
- Connection-management complexity
- Requires reconnection and replay support

For MVP:

```
Polling
```

Phase two:

```
SSE plus optional webhooks
```

Storing full payloads in the database versus object storage

Use the database for:

- Status
- Searchable metadata
- Keys
- Checksums
- Object references

Use object storage for:

- Large prompts
- Tool output
- Documents
- Generated reports
- Trace artifacts

This prevents database rows from becoming large and expensive.

Deliverable 5: Five-to-Seven-Minute Interview Script

Here is a spoken version you can repeatedly practise.

I will begin by clarifying the scope before choosing technologies.

The primary operation is to start a long-running agent workflow and track its lifecycle. For the MVP, I will support starting a run, getting its status, listing project runs, receiving tool callbacks, cancelling a run, storing artifact references and recording audit events. I will treat pause and resume, human approval, real-time token streaming and multi-region execution as phase-two capabilities.

I will assume this is a multi-tenant platform. Starting a run should return quickly, so workflow completion will be asynchronous. My initial latency targets are under 300 milliseconds at p95 for starting a run and under 200 milliseconds for status reads, with 99.9% availability. Accepted runs must not be silently lost, and duplicate client requests must not create duplicate runs.

For capacity, I will assume 100,000 daily active users and four runs per user per day, giving 400,000 runs per day. With one start request, six status reads and five tool callbacks per run, the service receives approximately 4.8 million API requests per day. That is about 56 average QPS, and using a ten-times peak factor, approximately 560 peak QPS. Structured run, tool and audit data grows by approximately 13 gigabytes per day before indexes. This traffic can be handled through horizontally scaled stateless APIs, while workflow execution and external tool concurrency are likely to be the more important scaling constraints.

The main API is POST `/v1/projects/{project_id}/runs`. It accepts an idempotency key and returns `202 Accepted` with a run ID and `QUEUED` status. Other APIs retrieve a run, list project runs using cursor pagination, cancel a run and receive signed tool callbacks. I use idempotency keys for starting runs and unique callback event IDs for deduplication.

For the data model, the main entities are Tenant, User, Project, AgentRun, ToolExecution, Artifact, Feedback and AuditLog. AgentRun contains the tenant, project, agent version, status and lifecycle timestamps. ToolExecution stores each tool call, attempt and outcome. Large inputs and outputs are stored in object storage, while the relational database stores searchable metadata and object references.

For the architecture, clients enter through an API gateway that performs authentication, rate limiting and routing. The AgentRun API validates the request and writes the new AgentRun, an audit event and a run-requested outbox event in one PostgreSQL transaction. An outbox publisher sends the event to a durable queue. This avoids losing a run when the database write succeeds but message publication temporarily fails.

Workflow workers consume run events, claim runs and execute the workflow. They persist checkpoints and update the run through a controlled state machine. Tool calls pass through a Tool Gateway that handles authorization, credentials, timeouts, quotas and audit logging. Asynchronous tool callbacks are authenticated, deduplicated and sent back to the workflow queue so execution can resume.

The message broker provides at-least-once delivery, so workers and callback handlers must be idempotent. I would use optimistic concurrency on the run version to prevent two workers from applying conflicting state transitions. Transient failures are retried using exponential backoff and jitter, while permanent failures go to a dead-letter queue for investigation.

For status reads, Redis can cache active-run state, while PostgreSQL remains the system of record. Status may be eventually consistent for a brief period, but run creation and lifecycle transitions require stronger consistency. Cursor pagination uses project ID, creation time and run ID so it remains stable and efficient as the table grows.

Security is tenant-aware at every layer. The authenticated identity determines the tenant, database queries include tenant ownership, tool credentials come from a secrets manager, callbacks are signed, and artifact access uses short-lived authorization. Sensitive tool output must not be written directly into logs.

For observability, I correlate request ID, run ID and tool-execution ID across structured logs and distributed traces. Important metrics include queue age, run duration, failure rate by agent version, tool latency, tool failure rate, retry count, LLM token usage and cost per run.

My initial design uses PostgreSQL because transactions, idempotency and state transitions are important. As scale grows, I can partition run data, move high-volume audit events to an append-oriented store and add SSE or webhooks for real-time status. The core design prioritizes reliable acceptance, durable asynchronous execution, tenant isolation and recoverable workflow state.

Practise the script until you can explain the same design without memorizing individual sentences.

Twelve Common Interview Mistakes

Requirements mistakes

1. **Drawing architecture immediately** First establish actors, scope, scale and NFRs.
2. **Not separating MVP from future features** The design becomes unnecessarily large.
3. **Ignoring asynchronous behavior** Long-running work should not be treated like a normal synchronous CRUD operation.
4. **Missing lifecycle states** A status field without defined transitions leads to unclear behavior.

Capacity mistakes

1. **Giving numbers without stating assumptions** Assumptions are more important than fake precision.
2. **Calculating only average QPS** Systems fail at peaks, not averages.
3. **Ignoring internal fan-out** One run may create several LLM calls, tool calls and audit events.
4. **Counting only raw storage** Include indexes, metadata, replication, backups and retention.
5. **Ignoring payload and artifact sizes** AI outputs, documents and traces can dominate storage and bandwidth.

API mistakes

1. **No idempotency for create operations** Client retries can create duplicate workflows.
2. **Using offset pagination for large changing datasets** Prefer cursor or keyset pagination.
3. **Returning inconsistent errors or internal stack traces** Use stable error codes, safe messages and a correlation ID.

Additional architecture mistake to remember:

> Adding Kafka, Redis, Elasticsearch and Kubernetes without demonstrating why they are needed is not a sign of seniority.

Ten Common Interview Questions and Answers

1. Why return `202 Accepted`?

Because starting a run and completing it are separate operations. The service acknowledges durable acceptance quickly while workers execute the workflow asynchronously.

---

2. How do you ensure an accepted run is not lost?

Write the AgentRun and an outbox event in the same database transaction. A background publisher retries until the event reaches the queue.

---

3. What happens when the queue delivers a message twice?

The consumer checks the current run status and version before applying work. Operations use unique event IDs or idempotency records so duplicate delivery does not duplicate the business action.

---

4. How do you handle duplicate tool callbacks?

Each callback includes a unique `event_id`. Store it under a unique constraint. A repeated event returns success without applying the state transition again.

---

5. Why use PostgreSQL?

Run creation and transitions benefit from transactions, unique constraints, relationships and flexible indexes. PostgreSQL is a strong starting point until measured scale requires a different partitioning model.

---

6. How do you stop two workers processing the same run?

Use a lease, conditional update or optimistic locking:

```
UPDATE agent_runs
SET status = 'RUNNING',
    version = version + 1
WHERE run_id = :run_id
  AND status = 'QUEUED'
  AND version = :expected_version;
```

Only one worker should successfully update the row.

---

7. Is run status strongly consistent?

The database is authoritative and strongly consistent for transitions. A Redis-backed status endpoint may briefly serve stale data, provided the product allows eventual consistency for reads.

---

8. How would you handle a noisy tenant?

Apply tenant-aware:

- Rate limits
- Concurrent-run quotas
- Queue partitions or weighted scheduling
- Token and tool budgets
- Storage limits

- Circuit breakers

This prevents one tenant from consuming all platform capacity.

---

9. How would you support multi-region execution?

A practical progression is:

1. Region-local APIs and workers
2. Tenant assigned to a home region
3. Region-local database writes
4. Globally replicated configuration
5. Cross-region backups
6. Controlled regional failover

Active-active writes require much more complicated conflict handling and should only be added for a clear requirement.

---

10. How do you control unsafe agent tools?

Put tools behind a policy-enforcing gateway. Validate tenant permission, tool allowlist, operation type, input schema, timeout, credentials and approval requirements. Do not allow the model to directly execute arbitrary commands.

Reusable System Design Skeleton

Use this on the interview whiteboard.

```
1. Clarify
   - Users and primary flow
   - MVP and out of scope
   - Sync, async or streaming
   - Consistency and lifecycle
   - Latency and availability
   - Multi-tenancy and compliance

2. Estimate
   - DAU × actions/day
   - Requests/day ÷ 86,400
   - Peak factor
   - Payload × QPS
   - Storage/day × retention
   - Internal fan-out
   - Cache-hit assumption

3. APIs
   - Main create endpoint
   - Read and list endpoints
   - Updates/actions
   - Idempotency
```

```
   - Cursor pagination
   - Standard errors
   - Authentication

4. Data model
   - Entities and relationships
   - Primary and foreign keys
   - Tenant ownership
   - Status and timestamps
   - Top queries
   - Indexes
   - Blob versus metadata storage

5. Architecture
   - Client and gateway
   - Stateless API service
   - System-of-record database
   - Cache
   - Queue and workers
   - Object storage
   - External integrations
   - Audit and observability

6. Deep dive
   - Critical request flow
   - State machine
   - Transactions
   - Idempotency
   - Concurrency
   - Failure recovery
   - Security
   - Scaling

7. Finish
   - Restate architecture
   - Explain reliability guarantee
   - Mention major trade-off
   - Explain scaling path
   - Identify phase-two features
```

One-sentence memory aid

```
Clarify what to build, estimate how large it is, define the contracts and
data,
draw the flow, then prove it survives failures.
```